

Capgemini Sprint Programme

Detailed Study Notes Based on Shiv-Assignments

Workspace analysed: D:\Capgemini\Shiv-Assignments

Generated: 2026-05-04

This PDF turns the week-by-week assignment folders into a revision guide. It covers the topics present in the workspace and adds closely related topics that are worth preparing before assessments, interviews, and practical labs.

How to Use These Notes

- Start with the folder analysis to understand the learning arc from web basics to Azure and APIs.
- Revise the core sections topic by topic, then implement the practice prompts in a small app.
- Use the missing-topic checklist to fill gaps before Sprint evaluation or technical discussion.
- Use the interview Q&A as quick oral revision; answer aloud without reading, then compare.

Repository Scan Summary

Analysed non-generated files: 1435. Ignored .git, bin, obj, build, .vs, .dotnet-home, packages, and wwwroot/lib vendor folders.

Extension	Count
.cs	496
.cshtml	389
.json	148
.css	78
.js	45
.csproj	45
.ico	38
.jpg	36
.slnx	32
.png	26
.jpeg	24
.html	23

Folder-by-Folder Analysis

This table covers the visible week/date folders and maps them to study topics.

Week	Date	What is in the folder	Study focus
WEEK-1	MARCH-7	Static site assets, CSS, images, JavaScript vendor files.	HTML document structure, CSS linking, static asset organization, browser-side page setup.
WEEK-1	MARCH-9	CSS exercises.	Selectors, styling rules, page layout basics, separation of style from markup.
WEEK-1	MARCH-10	HTML pages plus JavaScript for forms, arrays, and regex.	DOM access, form validation, arrays, email validation with regular expressions.
WEEK-1	MARCH-11	MVC_INTRODUCTION project.	ASP.NET Core MVC structure, controllers, views, models, routing, ViewBag/ViewData, simple action results.
WEEK-1	MARCH-12	Expanded MVC_INTRODUCTION project.	Strongly typed views, passing single and multiple objects, employee/department view models, LINQ filtering.
WEEK-1	MARCH-13	CRUD_OPERATIONS and DOG-MANAGEMENT-APP.	MVC CRUD screens, EF Core setup, SQLite/SQL Server packages, Razor forms, image upload, search.
WEEK-1	MARCH-14	DOG-MANAGEMENT-APP with migrations.	EF Core migrations, persisted CRUD flow, model validation, file handling, Delete/Edit patterns.
WEEK-2	MARCH-16	CodeFirst, DBFirst, MARCH-16, Student_Course_Management_System.	EF Core Code First, Database First, DbContext, products, Northwind, student-course-enrollment relationships.
WEEK-2	MARCH-18	DBFirst, LayoutAndSection, WEB-APPLICATION.	Scaffolded MVC CRUD, layouts, sections, department/employee management, Northwind queries.
WEEK-2	MARCH-19	CodeFirst, INVOICE-APPLICATION, SETUP-APPLICATION.	Data annotations, entity relationships, invoice/order domain modeling, customer-product-line-item flows.

Week	Date	What is in the folder	Study focus
WEEK-2	MARCH-20	CodeFirst, PARTIAL-VIEWS, ROUTING.	Partial views, conventional routing, student routes, repository interface pattern, post model exercises.
WEEK-3	MARCH-23	JQUERY and STATE-MANAGEMENT.	jQuery selectors/events, TempData, Session, Cookies, HttpContext, stateless web concepts.
WEEK-3	MARCH-24	WEBAPI-DEMO.	ASP.NET Core Web API, API controllers, EF Core, Swagger/OpenAPI, employee endpoints.
WEEK-3	MARCH-25	EMPLOYEE-MANAGEMENT-SYS.	Service layer, REST endpoints, pagination, file upload with IFormFile, validation, response typing.
WEEK-3	MARCH-26	Employee management continuation and MARCH-26 app.	API refinement, service abstraction, DTO/request classes, MVC/Razor practice.
WEEK-3	MARCH-27	WebApiInAsp.	Employee API plus MVC UI client, DTOs, EF Core, Swagger, ClosedXML package for Excel-oriented work.
WEEK-3	MARCH-28	PRODUCT-MANAGEMENT-SYSTEM and PRODUCT-UI.	Full-stack product CRUD, JavaScript fetch, API-backed UI, validation and search.
WEEK-4	MARCH-30	WebApiInAsp.	Web API with employee services, DTOs, EF Core, Identity/JWT groundwork.
WEEK-4	MARCH-31	WebApiInAsp continuation.	Authentication and employee API refinement, DI, Swagger, SQL Server persistence.
WEEK-4	APRIL-1	AZURE-MVC and WebApiInAsp.	Azure Blob Storage, containers/blobs, metadata, Identity, JWT, role policies, Swagger security.
WEEK-4	APRIL-2	WebApiInAsp.netcoreMvcDemo.	ASP.NET Core Web API + MVC, Identity, JWT bearer authentication, role-based endpoints.
WEEK-4	APRIL-3	AZURE and WEB-APPLICATION.	Azure portal practice, deployment/cloud service exposure, basic MVC web application review.
WEEK-5	APRIL-6	WEB-APPLICATION with AzureContext and screenshots.	Azure SQL style persistence, EF migrations, Person CRUD in MVC.
WEEK-5	APRIL-8	Day64 ProductAPIClient.	Consuming APIs from MVC, configurable API base URL, client-side product display.
WEEK-5	APRIL-9	AZURE-FUNCTIONS and CONSOLE-APP.	Azure Functions isolated worker, HTTP trigger, logging, simple console project.
WEEK-5	APRIL-10	AzureFunctionExample and TangyAzureFunc.	MVC client calling an Azure Function, JSON serialization, queue output binding.
WEEK-5	APRIL-11	Azure Function continuation.	Function app repetition/refinement, queue-bound sales request workflow, Application Insights packages.
WEEK-6	APRIL-13	AzureSpookyLoginApp.	MVC app calling external/function endpoint with HttpClient and Newtonsoft.Json.
WEEK-6	APRIL-14	INTRODUCTION text placeholder.	Review point for Azure/cloud theory and recap notes.
WEEK-6	APRIL-15	KEY-VAULT project.	Azure Key Vault concept, secret storage, configuration security, managed identity preparation.
WEEK-6	APRIL-16	COSMOS-DB-MVC-APPLICATION.	Azure Cosmos DB SDK, containers, partition keys, SQL-like queries, MVC CRUD over NoSQL data.
WEEK-6	APRIL-17	FILE text placeholder.	Review/submission placeholder for cloud module work.
WEEK-7	APRIL-20	Soft skills and personality development assessment.	Communication, assessment readiness, workplace behavior, self-presentation.
WEEK-7	APRIL-21	Image submissions/materials.	Evidence/screenshots or visual records; treat as task-submission artifacts.
WEEK-7	APRIL-22	Image submissions/materials.	Evidence/screenshots or visual records; review alongside trainer instructions if available.
WEEK-7	APRIL-23	Image submissions/materials.	Evidence/screenshots or visual records; likely non-code practical work.
WEEK-7	APRIL-24	INFO text about Git commands.	Git status, add, commit, branch, push, pull, merge, repository publishing workflow.
WEEK-7	APRIL-25	INFO text about Git commands.	Git command revision and GitHub upload workflow.
WEEK-8	APRIL-1	README with May 1 note.	Administrative/date note; no major code content found.

Week	Date	What is in the folder	Study focus
WEEK-8	APRIL-27	Azure VNet and VM setup notes.	Virtual networks, subnets, NSGs, VM deployment, private connectivity, nested virtualization.
WEEK-8	APRIL-28	APRIL28-PROJECT WEB-API.	Employee API, DTOs, IdentityDbContext, role seeding, EF Core SQL Server, image uploads.
WEEK-8	APRIL-29	APRIL29-PROJECT employee API frontend.	MVC frontend consuming API via HttpClientFactory, multipart forms, CRUD screens, export call.
WEEK-8	APRIL-30	LibraryManagementAPI.	Database-first EF model, authors/books/categories/members, borrowing and returning workflow APIs.

Detected .NET Project Matrix

Project	Target	Technology clues	Path
MVC_INTRODUCTION	net10.0	ASP.NET Core, MVC/Razor	WEEK-1\MARCH-11\MVC_INTRODUCTION\MVC_INTRODUCTION.csproj
MVC_INTRODUCTION	net10.0	ASP.NET Core, MVC/Razor	WEEK-1\MARCH-12\MVC_INTRODUCTION\MVC_INTRODUCTION.csproj
CRUD_OPERATIONS	net10.0	ASP.NET Core, EF Core, SQL Server, SQLite, MVC/Razor	WEEK-1\MARCH-13\CRUD_OPERATIONS\CRUD_OPERATIONS.csproj
DOG-MANAGEMENT-APP	net10.0	ASP.NET Core, EF Core, SQL Server, SQLite, MVC/Razor	WEEK-1\MARCH-13\DOG-MANAGEMENT-APP\DOG-MANAGEMENT-APP.csproj
DOG-MANAGEMENT-APP	net10.0	ASP.NET Core, EF Core, SQL Server, SQLite, MVC/Razor	WEEK-1\MARCH-14\DOG-MANAGEMENT-APP\DOG-MANAGEMENT-APP.csproj
CodeFirst	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite	WEEK-2\MARCH-16\CodeFirst\CodeFirst.csproj
DBFirst	net8.0	ASP.NET Core, EF Core, SQL Server	WEEK-2\MARCH-16\DBFirst\DBFirst.csproj
MARCH-16	net8.0	ASP.NET Core, EF Core, SQL Server	WEEK-2\MARCH-16\MARCH-16\MARCH-16.csproj
Student_Course_Management_System	net8.0	ASP.NET Core, EF Core, SQL Server	WEEK-2\MARCH-16\Student_Course_Management_System\Student_Course_Management_System.csproj
DBFirst	net8.0	ASP.NET Core, EF Core, SQL Server	WEEK-2\MARCH-18\DBFirst\DBFirst.csproj
LayoutAndSection	net8.0	ASP.NET Core	WEEK-2\MARCH-18\LayoutAndSection\LayoutAndSection.csproj
WEB-APPLICATION	net8.0	ASP.NET Core, EF Core, SQL Server, MVC/Razor	WEEK-2\MARCH-18\WEB-APPLICATION\WEB-APPLICATION.csproj
CodeFirst	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite	WEEK-2\MARCH-19\CodeFirst\CodeFirst.csproj
INVOICE-APPLICATION	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, MVC/Razor	WEEK-2\MARCH-19\INVOICE-APPLICATION\INVOICE-APPLICATION.csproj
SETUP-APPLICATION	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, MVC/Razor	WEEK-2\MARCH-19\SETUP-APPLICATION\SETUP-APPLICATION.csproj
CodeFirst	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite	WEEK-2\MARCH-20\CodeFirst\CodeFirst.csproj
PARTIAL-VIEWS	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, MVC/Razor	WEEK-2\MARCH-20\PARTIAL-VIEWS\PARTIAL-VIEWS.csproj
ROUTING	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite	WEEK-2\MARCH-20\ROUTING\ROUTING.csproj
STATE-MANAGEMENT	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite	WEEK-3\MARCH-23\STATE-MANAGEMENT\STATE-MANAGEMENT.csproj
WEBAPI-DEMO	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, Swagger, Web API	WEEK-3\MARCH-24\WEBAPI-DEMO\WEBAPI-DEMO.csproj
EMPLOYEE-MANAGEMENT-SYS	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, Swagger	WEEK-3\MARCH-25\EMPLOYEE-MANAGEMENT-SYS\EMPLOYEE-MANAGEMENT-SYS.csproj
EMPLOYEE-MANAGEMENT-SYS	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, Swagger	WEEK-3\MARCH-26\EMPLOYEE-MANAGEMENT-SYS\EMPLOYEE-MANAGEMENT-SYS.csproj
MARCH-26	net8.0	ASP.NET Core	WEEK-3\MARCH-26\MARCH-26\MARCH-26.csproj

Project	Target	Technology clues	Path
WebApiInAsp	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, Swagger, Excel/ClosedXML, Web API	WEEK-3\MARCH-27\WebApiInAsp\WebApiInAsp.csproj
PRODUCT-MANAGEMENT-SYSTEM	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, Swagger	WEEK-3\MARCH-28\PRODUCT-MANAGEMENT-SYSTEM\PRODUCT-MANAGEMENT-SYSTEM.csproj
AZURE-MVC	net8.0	ASP.NET Core, Azure Blob Storage, MVC/Razor	WEEK-4\APRIL-1\AZURE-MVC\AZURE-MVC.csproj
WebApiInAsp	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, Identity, JWT, Swagger	WEEK-4\APRIL-1\WebApiInAsp\WebApiInAsp.csproj
WebApiInAsp.netcoreMvcDemo	net8.0	ASP.NET Core, EF Core, SQL Server, Identity, JWT, Swagger, Excel/ClosedXML	WEEK-4\APRIL-2\WebApiInAsp.netcoreMvcDemo\WebApiInAsp.netcoreMvcDemo.csproj
WEB-APPLICATION	net8.0	ASP.NET Core, MVC/Razor	WEEK-4\APRIL-3\WEB-APPLICATION\WEB-APPLICATION.csproj
WebApiInAsp	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, Identity, JWT, Swagger	WEEK-4\MARCH-30\WebApiInAsp\WebApiInAsp.csproj
WebApiInAsp	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, Identity, JWT, Swagger	WEEK-4\MARCH-31\WebApiInAsp\WebApiInAsp.csproj
AzureFunctionTangyWeb	net8.0	ASP.NET Core, Newtonsoft.Json	WEEK-5\APRIL-10\AzureFunctionExample\AzureFunctionTangyWeb\AzureFunctionTangyWeb.csproj
TangyAzureFunc	net8.0	Azure Functions, Azure Queue, Newtonsoft.Json, MVC/Razor	WEEK-5\APRIL-10\TangyAzureFunc\TangyAzureFunc\TangyAzureFunc.csproj
AzureFunctionTangyWeb	net8.0	ASP.NET Core, Newtonsoft.Json	WEEK-5\APRIL-11\AzureFunctionExample\AzureFunctionTangyWeb\AzureFunctionTangyWeb.csproj
TangyAzureFunc	net8.0	Azure Functions, Azure Queue, Newtonsoft.Json, MVC/Razor	WEEK-5\APRIL-11\TangyAzureFunc\TangyAzureFunc\TangyAzureFunc.csproj
WEB-APPLICATION	net8.0	ASP.NET Core, EF Core, SQL Server, SQLite, MVC/Razor	WEEK-5\APRIL-6\WEB-APPLICATION\WEB-APPLICATION.csproj
ProductAPIClient	net8.0	ASP.NET Core, Web API	WEEK-5\APRIL-8\Day64\ProductAPIClient\ProductAPIClient.csproj
AZURE-FUNCTIONS	net8.0	Azure Functions, MVC/Razor	WEEK-5\APRIL-9\AZURE-FUNCTIONS\AZURE-FUNCTIONS.csproj
CONSOLE-APP	net8.0	ASP.NET/Core practice	WEEK-5\APRIL-9\CONSOLE-APP\CONSOLE-APP.csproj
AzureSpookyLoginApp	net8.0	ASP.NET Core, Newtonsoft.Json	WEEK-6\APRIL-13\AzureSpookyLoginApp\AzureSpookyLoginApp.csproj
KEY-VAULT	net8.0	ASP.NET/Core practice	WEEK-6\APRIL-15\KEY-VAULT\KEY-VAULT.csproj
COSMOS-DB-MVC-APPLICATION	net8.0	ASP.NET Core, Cosmos DB, Newtonsoft.Json, MVC/Razor	WEEK-6\APRIL-16\COSMOS-DB-MVC-APPLICATION\COSMOS-DB-MVC-APPLICATION.csproj
WEB-API	net8.0	ASP.NET Core, EF Core, SQL Server, Identity, JWT, Swagger, Web API	WEEK-8\APRIL-28\APRIL28-PROJECT\WEB-API\WEB-API.csproj
EMPLOYEE-API-FRONTEND	net8.0	ASP.NET Core, Web API	WEEK-8\APRIL-29\APRIL29-PROJECT\EMPLOYEE-API-FRONTEND\EMPLOYEE-API-FRONTEND.csproj
LibraryManagementAPI	net8.0	ASP.NET Core, EF Core, SQL Server, Swagger, Web API	WEEK-8\APRIL-30\LibraryManagementAPI\LibraryManagementAPI\LibraryManagementAPI.csproj

Learning Roadmap

Stage	Theme	What to master
Stage 1	Web basics	HTML forms, CSS, JavaScript validation, arrays, regex, jQuery, fetch.
Stage 2	MVC	Controllers, actions, Razor views, layouts, partials, routing, model binding, validation.
Stage 3	Data access	EF Core, SQL Server, SQLite, Code First, DB First, migrations, relationships, LINQ.
Stage 4	CRUD systems	Dog, product, employee, invoice, person, and student/course applications.
Stage 5	Web API	REST endpoints, DTOs, Swagger, status codes, pagination, search, file upload.

Stage	Theme	What to master
Stage 6	Security	Identity, JWT bearer tokens, roles, policies, secrets, authenticated API calls.
Stage 7	Azure	Blob Storage, Functions, Queues, Key Vault, Cosmos DB, Azure SQL, VNet/VM/NSG.
Stage 8	Professional workflow	Git/GitHub, review habits, deployment readiness, testing, logging.

1. Web Fundamentals: HTML, CSS, JavaScript, Regex, jQuery

Week 1 starts with static web basics before ASP.NET Core. These are the foundations behind Razor views, MVC forms, and API-backed pages.

Study Notes

- HTML gives structure: forms, inputs, buttons, labels, tables, links, images, and script/style references.
- CSS controls presentation: selectors, specificity, box model, spacing, colors, positioning, responsive layout, and Bootstrap utility classes.
- JavaScript adds behavior: functions, variables, arrays, DOM lookup, event handlers, conditions, loops, and async fetch calls.
- Regex is useful for input checks such as email validation. Know anchors (^ and \$), character classes, quantifiers, escaping, and test methods.
- jQuery is older but still appears in MVC templates and validation packages. Know selectors, event binding, hide/show, attr/css, and AJAX patterns.

Where It Appears in Your Folders

- WEEK-1/MARCH-10 has form validation and email regex examples.
- WEEK-3/MARCH-23/JQUERY has jQuery practice pages.
- WEEK-3/MARCH-28/PRODUCT-UI/app.js uses fetch, JSON, DOM card creation, edit mode, delete calls, and client-side search.

Watch Points

- Client-side validation improves experience but does not replace server-side validation.
- When checking invalid zip code, the usual condition is empty OR wrong length. Empty AND wrong length can miss some invalid cases.
- Never trust file names, file extensions, form values, or client JSON without server validation.

Practice Tasks

- Build a form with name, email, phone, and country fields; validate it with JavaScript and then repeat validation on the server.
- Use fetch to call a simple GET endpoint, render cards, then add POST/PUT/DELETE buttons.
- Write regex patterns for email, phone number, pincode, and simple password rules.

2. ASP.NET Core MVC Fundamentals

Most early projects are MVC applications. MVC separates request handling, data shape, and UI rendering.

Study Notes

- Model: classes that represent data or view data. Examples include Employee, Department, Dog, Product, Customer, Invoice, and view models.
- View: Razor .cshtml templates. They use @model, tag helpers, layout pages, partials, forms, and server-rendered HTML.

- Controller: classes ending in Controller. Actions return IActionResult, ViewResult, RedirectToAction, NotFound, File, JSON, or strings.
- Program.cs configures services and middleware: AddControllersWithViews, AddDbContext, UseStaticFiles, UseRouting, UseAuthentication, UseAuthorization, MapControllerRoute.
- Conventional route pattern usually looks like {controller=Home}/{action=Index}/{id?}. Attribute routing is common in Web API.
- Model binding maps form fields, query string values, route values, and files into action parameters or model objects.
- Validation uses data annotations, ModelState.IsValid, validation tag helpers, and _ValidationScriptsPartial for client-side unobtrusive validation.
- Anti-forgery tokens protect POST forms from CSRF attacks. MVC scaffolded forms include [ValidateAntiForgeryToken].

Where It Appears in Your Folders

- WEEK-1/MARCH-11 and MARCH-12 show controller actions, ViewBag, ViewData, strongly typed views, and employee/department models.
- WEEK-1/MARCH-13 and MARCH-14 show MVC CRUD screens for dog management.
- WEEK-2/MARCH-18/LayoutAndSection and WEEK-2/MARCH-20/PARTIAL-VIEWS focus on layout, sections, and reusable UI fragments.

Watch Points

- Prefer strongly typed view models over ViewBag/ViewData for real apps; they are easier to validate and refactor.
- Use RedirectToAction after successful POST to avoid duplicate form submission on browser refresh.
- Keep controllers thin. Move business rules to services when logic grows.

Practice Tasks

- Explain the full flow of a POST form: browser form -> route -> controller action -> model binding -> ModelState -> database -> redirect.
- Create a view model that combines Employee and Department and render it in a strongly typed Razor view.
- Add a partial view for a repeated employee card and render it from an index page.

3. Entity Framework Core and Database Work

Weeks 1 and 2 move heavily into EF Core, SQL Server, SQLite, Code First, Database First, migrations, and relationships.

Study Notes

- DbContext represents a database session. DbSet<T> represents a table or collection of entities.
- Code First starts from C# classes and creates database schema through migrations.
- Database First starts from an existing database and scaffolds models/DbContext.
- Migrations capture schema changes. Core commands: dotnet ef migrations add Name, dotnet ef database update, dotnet ef migrations remove.
- Use async database calls: ToListAsync, FindAsync, FirstOrDefaultAsync, SaveChangesAsync.
- Use AsNoTracking for read-only queries to reduce change-tracking overhead.
- Use Include and ThenInclude to load related data such as Book -> Author and Book -> Category.
- Data annotations include [Key], [Required], [Range], [Column(TypeName=...)], [DatabaseGenerated], [NotMapped].
- Relationships: one-to-many for Author -> Books, Category -> Books, Member -> BorrowRecords; many-to-many often needs a join entity.

Where It Appears in Your Folders

- WEEK-2/MARCH-16 compares CodeFirst and DBFirst, and includes Student_Course_Management_System.
- WEEK-2/MARCH-19/INVOICE-APPLICATION models Customer, Product, Invoice, and InvoiceLineItem.
- WEEK-8/APRIL-30/LibraryManagementAPI uses a database-first ApplicationDbContext with relationships for books, authors, categories, members, and borrow records.

Watch Points

- Do not hard-code production connection strings in source files. Use appsettings, user secrets, environment variables, or Key Vault.
- Avoid overposting by binding only allowed properties or using DTO/view model classes.
- For important multi-step updates such as borrow/return, consider transactions and concurrency handling.

Practice Tasks

- Design a Student-Course-Enrollment schema and explain keys and navigation properties.
- Write LINQ queries for filtering, sorting, pagination, grouping, and joins.
- Create a migration, inspect the generated code, apply it, and explain Up and Down methods.

4. CRUD Application Patterns

Dog, Product, Employee, Person, Invoice, and Library assignments all repeat CRUD. Mastering one clean CRUD flow helps with nearly every Sprint task.

Study Notes

- Create: GET action returns an empty form; POST validates input, adds entity, saves changes, redirects.
- Read: Index lists records; Details loads one record and returns NotFound if missing.
- Update: GET loads existing record; POST checks route id matches model id, validates, updates, catches concurrency issues, redirects.
- Delete: GET confirms; POST deletes and redirects. In APIs, DELETE returns NoContent or deleted DTO.
- Search and pagination should be applied beforeToListAsync so filtering happens in the database.
- File upload CRUD needs upload directory creation, unique names, size/type validation, and old-file cleanup on update/delete.

Where It Appears in Your Folders

- WEEK-1/MARCH-14/DOG-MANAGEMENT-APP shows image upload and in-memory list CRUD.
- WEEK-3/MARCH-25 employee API adds pagination and safer image upload validation.
- WEEK-8/APRIL-28 employee API and WEEK-8/APRIL-29 frontend show API plus MVC-client CRUD with multipart forms.

Watch Points

- In-memory static lists reset when the app restarts. Use a database for persistent data.
- Do not expose wwwroot upload paths unless you intentionally want public files.
- When deleting uploaded files, ensure the path stays inside the intended upload folder.

Practice Tasks

- Implement product CRUD once in MVC and once as a Web API.
- Add search and pagination to an index endpoint.
- Add server-side file validation for extension, content type, and max size.

5. State Management in ASP.NET Core

HTTP is stateless, so Week 3 introduces practical ways to carry small pieces of state between requests.

Study Notes

- ViewBag/ViewData last for the current request only.
- TempData survives one redirect/request and is useful for success/error messages. TempData.Keep preserves it longer.
- Session stores data per user session on the server, referenced by a session cookie.
- Cookies store small values in the browser and are sent with requests. Use security flags for sensitive scenarios.
- Static fields and instance fields in controllers are not reliable per-user state. Controllers are created per request, and static data is shared.

Where It Appears in Your Folders

- WEEK-3/MARCH-23/STATE-MANAGEMENT uses TempData, Session, Cookie comments, and HttpContext.
- The LoginController demonstrates session write/remove patterns, even though the Welcome condition should be reviewed carefully.

Watch Points

- Session is convenient but can hide scaling problems in distributed deployments.
- Never store passwords, tokens, or sensitive data in plain cookies.
- For authenticated identity, use ASP.NET Core Identity or JWT rather than hand-made session checks.

Practice Tasks

- Create a flash message with TempData after a POST redirect.
- Store a username in Session, display it, then clear it on logout.
- Explain the difference between authentication state and arbitrary session state.

6. ASP.NET Core Web API and REST

Weeks 3, 4, and 8 focus on API projects that expose data to JavaScript clients, MVC clients, and Swagger.

Study Notes

- [ApiController] enables automatic model validation behavior, binding source inference, and better error responses.
- [Route("api/[controller]")] maps controller names to API paths. [HttpGet], [HttpPost], [HttpPut], [HttpDelete] declare operations.
- Use ControllerBase for APIs; use Controller when you also return views.
- Return appropriate status codes: 200 OK, 201 Created, 204 NoContent, 400 BadRequest, 401 Unauthorized, 403 Forbidden, 404 NotFound, 409 Conflict.
- DTOs keep API contracts separate from EF entities. This reduces overposting and lets you control response shape.
- Swagger/OpenAPI documents endpoints and allows test calls from the browser.
- IHttpClientFactory creates configured HttpClient instances for calling external APIs from MVC apps.
- CORS is needed when a browser-based frontend calls an API from a different origin.

Where It Appears in Your Folders

- WEEK-3/MARCH-24/WEBAPI-DEMO introduces API controllers and Swagger.
- WEEK-3/MARCH-28 combines PRODUCT-MANAGEMENT-SYSTEM with PRODUCT-UI fetch calls.
- WEEK-8/APRIL-29 uses IHttpClientFactory, multipart form data, JSON deserialization, and MVC views over an API.

Watch Points

- Use CreatedAtAction for successful POST when the new resource has a URL.
- Do not return EF entities with large circular navigation graphs; shape responses with DTOs.
- Validate page and pageSize to avoid negative Skip values or huge queries.

Practice Tasks

- Design a REST API for books with GET all, GET by id, POST, PUT, DELETE, and search.
- Add Swagger annotations or response types to document success and error cases.
- Consume your API from an MVC frontend and from a plain HTML/JS frontend.

7. Authentication, Authorization, Identity, and JWT

Week 4 and Week 8 introduce IdentityDbContext, roles, JWT bearer tokens, logout, and policy-based authorization.

Study Notes

- Authentication answers: who is this user? Authorization answers: what can this user do?
- ASP.NET Core Identity provides users, roles, password hashing, logout, claims, and token providers.
- UserManager creates/finds users; RoleManager manages roles; SignInManager validates passwords/sign-in attempts.
- JWT tokens usually include subject/user id, name, email, roles, issuer, audience, expiry, and a signature.
- JWT bearer middleware validates issuer, audience, expiry, and signing key before setting HttpContext.User.
- Role-based authorization checks roles directly; policy-based authorization names rules such as AdminOnly or EmployeeWrite.
- Swagger can be configured with Bearer security so protected endpoints can be tested with a token.

Where It Appears in Your Folders

- WEEK-4/APRIL-1/WebApiInAsp has AuthService, JwtOptions, BootstrapAdminOptions, Identity seeding, roles, and policies.
- WEEK-8/APRIL-28/WEB-API uses IdentityDbContext<IdentityUser> and seeds Admin, User, and HR roles.

Watch Points

- JWT secrets must not be committed to source control. Store them in user secrets, environment variables, or Key Vault.
- Tokens should be short-lived; production systems often add refresh tokens.
- Use HTTPS for token-bearing requests.

Practice Tasks

- Trace login: credentials -> user lookup -> password check -> claims -> JWT -> client Authorization header -> protected endpoint.
- Create Admin and User roles and protect one endpoint with [Authorize(Roles = "Admin")].
- Explain 401 versus 403 in an interview.

8. Azure Storage, Functions, Key Vault, Cosmos DB, and Networking

Weeks 4, 5, 6, and 8 move from local apps into cloud services and deployment-style thinking.

Study Notes

- Azure Blob Storage stores files/objects inside containers. Use BlobServiceClient, BlobContainerClient, and BlobClient.

- Blob metadata stores custom key/value information such as title and comment. Public access should be controlled carefully.
- Azure Functions are event-driven. The repo uses isolated worker functions with HttpTrigger and QueueOutput.
- Queue output binding lets a function receive HTTP input and place a message into Azure Storage Queue for later processing.
- Azure Key Vault stores secrets, keys, and certificates. Apps should access it through managed identity where possible.
- Cosmos DB is globally distributed NoSQL. Important ideas: database, container, item, id, partition key, RU/s, consistency, query iterator.
- Azure Virtual Network gives private network space. Subnets divide it. NSGs control allowed inbound/outbound traffic.
- Azure VMs can have public and private IPs. VM-to-VM communication inside a VNet should use private IPs when possible.
- Nested virtualization means running a VM inside another Azure VM and needs supported VM sizes.

Where It Appears in Your Folders

- WEEK-4/APRIL-1/AZURE-MVC manages containers and blobs and reads metadata.
- WEEK-5/APRIL-9 to APRIL-11 contains HTTP-trigger functions and queue output.
- WEEK-6/APRIL-16/COSMOS-DB-MVC-APPLICATION performs MVC CRUD against Cosmos DB.
- WEEK-8/APRIL-27 has VNet, subnet, NSG, VM, VM-to-VM, and nested virtualization notes.

Watch Points

- Parameterized queries are safer than string interpolation in Cosmos DB search queries.
- Stop unused VMs and storage resources to avoid cloud cost surprises.
- Use least-privilege NSG rules. Do not expose ports that are not required.

Practice Tasks

- Upload a file to Blob Storage, attach metadata, list blobs, and delete one blob.
- Create an HTTP-trigger Azure Function that writes a message to a queue.
- Draw a VNet with two subnets, two VMs, an NSG, and allowed RDP/SSH rules.

9. Git, GitHub, and Professional Workflow

Week 7 explicitly mentions Git commands for uploading code to GitHub. This matters in every project team.

Study Notes

- git status shows changed files. git add stages changes. git commit records a snapshot. git push uploads commits.
- git clone copies a remote repository. git pull fetches and merges remote changes. git fetch only downloads remote data.
- Branches isolate work. Use git switch -c feature-name, commit locally, push, and open a pull request.
- git diff shows unstaged changes. git diff --staged shows staged changes.
- git log --oneline --graph helps understand history.
- .gitignore prevents generated files such as bin, obj, .vs, node_modules, and secrets from being committed.
- Avoid destructive commands unless you understand them: git reset --hard and force push can discard work.

Where It Appears in Your Folders

- WEEK-7/APRIL-24 and APRIL-25 INFO files mention Git commands and GitHub upload workflow.
- The current repository itself is a day-wise training record and can be used to practice commit hygiene.

Watch Points

- Do not commit secrets, connection strings, tokens, or large generated build folders.
- Commit messages should say what changed and why.
- Pull before pushing when collaborating to avoid conflicts.

Practice Tasks

- Create a branch, edit a small README, commit it, inspect diff/log, then merge it back.
- Resolve a simple merge conflict in a text file.
- Explain staging area versus working tree versus repository history.

10. Capstone Domain Modeling: Employee and Library APIs

Week 8 consolidates the earlier material into realistic API-backed systems.

Study Notes

- Employee API: DTOs, EF Core, SQL Server, image upload, search, pagination, and role seeding.
- Employee MVC frontend: consumes API using IHttpConnectionFactory, serializes/deserializes JSON, sends multipart forms, and renders CRUD views.
- Library API: Authors, Books, Categories, Members, BorrowRecords, navigation properties, Include queries, and borrow/return actions.
- Borrow flow: validate book exists, validate available copies, validate member exists, decrement copies, create borrow record, save.
- Return flow: validate borrow record exists, reject double return, load book, set return date/status, increment copies, save.

Where It Appears in Your Folders

- WEEK-8/APRIL-28 has WEB-API with employee services and DTOs.
- WEEK-8/APRIL-29 has EMPLOYEE-API-FRONTEND that calls the API.
- WEEK-8/APRIL-30 has LibraryManagementAPI with BooksController and BorrowController.

Watch Points

- Borrow/return should ideally use transactions to avoid copy-count inconsistencies.
- Directly setting EntityState.Modified can update all columns; DTO-based updates are safer.
- Database-first scaffolding may include warnings about connection strings. Move secrets out of source.

Practice Tasks

- Add a due date and fine calculation to BorrowRecord.
- Add search by book title/author/category and pagination.
- Create DTOs so API responses do not expose full EF navigation graphs.

Missing but Related Topics to Add to Prep

These topics are not equally visible in the folder contents, but they are strongly related to the programme path and common .NET/Azure assessment expectations.

Topic	Why it matters
Clean Architecture and SOLID	The repo uses controllers, DbContext, and some services. Study single responsibility, dependency inversion, service boundaries, and why controllers should stay thin.
Testing	Add xUnit tests for services, WebApplicationFactory integration tests for APIs, and mock-based tests for HttpClient/API clients.
Global Exception Handling	Learn middleware or filters that convert exceptions into consistent ProblemDetails responses.

Topic	Why it matters
Logging and Monitoring	Use ILogger, structured logs, Application Insights, log levels, correlation ids, and request tracing.
Configuration and Secrets	Use appsettings by environment, user secrets locally, environment variables in deployment, and Azure Key Vault for cloud secrets.
CORS and Browser Security	Know why browser clients need CORS, and how to restrict allowed origins/methods/headers.
Transactions and Concurrency	Use database transactions for borrow/return and RowVersion/concurrency tokens for simultaneous updates.
API Versioning	Learn URL/header versioning, backward compatibility, and deprecating old endpoints.
Validation Libraries	Data annotations are present; also know FluentValidation for larger validation rules.
Object Mapping	Manual DTO mapping is used; learn AutoMapper or explicit mapper classes and when each is appropriate.
SQL Performance	Study indexes, query plans, N+1 queries, Include cost, pagination strategy, and AsNoTracking.
Deployment and CI/CD	Add GitHub Actions, dotnet build/test, publish artifacts, Azure App Service deployment, and environment-specific configuration.
Docker Basics	Understand Dockerfile, image, container, ports, volumes, and running SQL Server locally in a container.
Security Hardening	HTTPS, secure cookies, authorization checks, upload scanning, rate limiting, password policy, token expiry, and least privilege.
Accessibility and Responsive UI	Use labels, keyboard navigation, focus states, color contrast, and responsive Bootstrap layouts.

Command Cheat Sheet

Task	Command
Run MVC/API app	dotnet run
Restore packages	dotnet restore
Build solution/project	dotnet build
Add EF migration	dotnet ef migrations add InitialCreate
Apply EF migration	dotnet ef database update
Remove last migration	dotnet ef migrations remove
Scaffold DB First example	dotnet ef dbcontext scaffold "<connection-string>" Microsoft.EntityFrameworkCore.SqlServer -o Models
Git status	git status
Stage files	git add .
Commit	git commit -m "message"
Create branch	git switch -c feature/topic
Push branch	git push -u origin feature/topic
View history	git log --oneline --graph --decorate

Key Code Patterns to Memorize

MVC POST pattern

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Model model)
{
    if (!ModelState.IsValid) return View(model);
    _context.Add(model);
    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
```

API GET by id pattern

```
[HttpGet("{id}")]
public async Task<ActionResult<Dto>> GetById(int id)
```

```
{
    var item = await _service.GetByIdAsync(id);
    return item is null ? NotFound() : Ok(item);
}
```

Pagination pattern

```
page = Math.Max(page, 1);
pageSize = Math.Clamp(pageSize, 1, 100);
var items = await query
    .Skip((page - 1) * pageSize)
    .Take(pageSize)
    .ToListAsync();
```

Safe upload outline

Validate extension and size.
 Create upload folder if missing.
 Generate a GUID file name.
 Copy stream to wwwroot/uploads.
 Store only the relative URL/path.
 Delete old file on update/delete when appropriate.

Interview and Viva Q&A

Question	Short answer
What is MVC?	MVC separates an app into Models for data, Views for UI, and Controllers for request handling and coordination.
What is the difference between ViewBag, ViewData, TempData, and Session?	ViewBag/ViewData live for one request. TempData survives redirects briefly. Session stores per-user server-side data across requests.
What is model binding?	ASP.NET Core maps route values, query strings, forms, headers, JSON bodies, and uploaded files into action parameters or model objects.
Why use DTOs?	DTOs protect domain/EF entities, avoid overposting, shape API responses, and keep contracts stable.
What does [ApiController] do?	It improves API behavior with automatic model validation responses, binding inference, and better client error responses.
Code First versus Database First?	Code First starts from C# classes and migrations. Database First scaffolds classes from an existing database.
What is DbContext?	It represents a session with the database and tracks entities, queries, and changes.
Why use async EF calls?	They free the request thread while waiting on I/O and improve scalability under load.
What is dependency injection?	A pattern where dependencies are supplied by the framework rather than manually constructed, improving testability and separation.
What is middleware?	Components in the request pipeline that can inspect, modify, handle, or pass requests and responses.
401 versus 403?	401 means not authenticated or invalid credentials. 403 means authenticated but not authorized.
What is JWT?	A signed token containing claims used by APIs to authenticate requests without server-side session lookup.
What are roles and policies?	Roles group users by responsibility; policies define named authorization rules, often based on roles or claims.
What is Swagger/OpenAPI?	A machine-readable API description with UI tooling for documentation and testing.
What is CORS?	A browser security policy that controls cross-origin API calls from frontend code.
What is Azure Blob Storage?	Object storage for files such as images, documents, and logs, grouped into containers.
What is an Azure Function?	A serverless function triggered by events such as HTTP requests, queues, timers, or blob changes.
What is Cosmos DB?	A globally distributed NoSQL database service with containers, partition keys, RUs, and multiple APIs.
What is a VNet?	A private network boundary in Azure that lets resources communicate securely using private IPs.
What is an NSG?	A Network Security Group that filters inbound and outbound traffic by source, destination, port, and protocol.

Question	Short answer
What does git add do?	It stages selected changes for the next commit.
What is a pull request?	A review workflow for proposing branch changes before merging them into a target branch.

Seven-Day Revision Plan

Day	Focus	Output
Day 1	Frontend + MVC basics	Revise HTML/CSS/JS, Razor, controllers, actions, routing, ViewBag/ViewData/TempData.
Day 2	EF Core	Build one Code First model, create migration, run CRUD, explain relationships and LINQ.
Day 3	MVC CRUD	Rebuild a Dog/Product style CRUD app with validation, search, and file upload.
Day 4	Web API	Create REST endpoints with DTOs, Swagger, pagination, search, and proper status codes.
Day 5	Security	Implement Identity/JWT login, roles, policies, and protected endpoints.
Day 6	Azure	Review Blob, Functions, Queue, Key Vault, Cosmos DB, VNet/VM/NSG and deployment concepts.
Day 7	Capstone + Git	Explain the employee and library APIs, run Git workflow practice, answer interview Q&A aloud.

Final Assessment Checklist

- I can explain MVC request flow and build a CRUD screen without copying.
- I can create an EF Core model, migration, relationship, and LINQ query.
- I can design REST endpoints and return correct status codes.
- I can explain DTOs, model binding, validation, and overposting.
- I can secure an endpoint with JWT and roles.
- I can describe Blob Storage, Azure Functions, Queues, Key Vault, Cosmos DB, VNet, VM, and NSG.
- I can use Git branch/add/commit/push/pull and explain a pull request.
- I can discuss missing production concerns: tests, logs, secrets, transactions, CI/CD, and deployment.